COI - Assignment 1

Solutions Document

Scenario - Grama Jyothi Rural Electricity Co-operative

Data types, constants, macros, headers, I/O, type conversion, compilation and testing in C

Student name	P. Ritika
Reg.no	1920260115
Course code	CSA0201
Date submitted	11-11-2026

Contents

Task	Topic	Programs / Evidence in this document
1	Understand the problem and identify the data	Data table, rules, assumptions, text-vs-integer analysis
2	Choose the data types	consumption() program + output screenshot, sizeof/limits, float failure
3	Define the constants	compute_bill() program + screenshots, RATE2 = 230 experiment
4	Use macros safely	Macro program + screenshot, gcc -E, pitfalls demo
5	Header files and conditional compilation	billing.h / billing.c / main, normal and -DDEBUG runs
6	Read and print data correctly	scanf and fgets/strtol programs + screenshots, wrong specifier test
7	Control type conversion and precision	Five conversion surprises with output
8	Trace the compilation process	gcc -E / -S / -c / link, three kinds of error, warnings
9	Implement and test the billing program	Complete program, screenshots, 11-case test table, 5-bill hand check
10	Compare expectations with execution	float / double / int / long long totals for 4,000 to 800,000 bills
11	Recommend and defend the final design	Recommendation table, tariff revision, limitations
12	Design challenge: re-evaluate and adapt	Impact analysis, smart-meter program with tariff file

Note on evidence: the screenshots labelled "OnlineGDB" are taken from my own runs on onlinegdb.com. Where a screenshot of the finished output was not available, the output shown in a black terminal block was produced by compiling and running the listed source with gcc 13.3 on Linux x86-64 (the same platform OnlineGDB uses), so sizes such as sizeof(long) = 8 apply to both.

TASK 1 - Understand the Problem and Identify the Data

1.1 Inputs and outputs

Data item	Kind	Meaning	Why it is stored this way
household number	input	Identifies the house (4,000 to 800,000 of them)	A label, never used in arithmetic. It must exceed 16 bits (800,000 > 65,535), so an unsigned 32-bit value or a string is needed.
category	input	Domestic / commercial / agricultural	A small code (one char or enum). Kept so a different tariff per category can be added later.
previous reading	input	Meter register at last visit	The register is a 16-bit counter, so 0 to 65,535. Never negative, never fractional.
current reading	input	Meter register now	Same range as above. May be smaller than previous after a roll-over.
units consumed	output	current - previous, corrected for roll-over	Whole units, never negative, at most 65,535 in one bill. Unsigned.
energy charge	output	Slab-wise charge for the units	Money: kept as an integer number of paise.
fixed charge	output / constant	Rs 50 per bill	5000 paise. The same for every household, so it is a constant.
total bill	output	fixed + energy	Integer paise; district total needs a wider type.

1.2 Values that never change during a run (constants)

- Slab limits: 100 units and 200 units.
- Rates: 0 paise (first slab), 225 paise (second slab), 450 paise (above 200).
- Fixed charge: 5000 paise.
- Register size: maximum reading 65,535 and roll-over modulus 65,536 (a hardware fact).
- Each of these is written once and named, so a tariff revision touches one line (Task 3).

1.3 Negative, fractional and very large values

- **Can be negative:** the raw difference current - previous becomes negative when the meter rolls over (e.g. 120 - 65400). It must be corrected, never billed. A bill itself can never be negative.
- **Fractional:** the rate Rs 2.25 and any rupee amount. Binary floating point cannot store 0.1 or 0.25-style decimals exactly in general, so money is stored as whole paise.
- **Very large:** the total for a district. 800,000 bills of a few hundred rupees each is about 3.3×10^{10} paise, far above the 2,147,483,647 limit of a 32-bit int (measured in Task 10).

1.4 Rules every correct bill must satisfy

- Bill >= fixed charge >= 0, so a bill is never negative.
- total = fixed charge + energy charge, and energy charge = slab 1 part + slab 2 part + slab 3 part.
- The units in the three slabs add up to the units consumed.
- The bill never decreases when units increase (monotonic); a slab boundary does not cause a jump.
- Every amount is an exact whole number of paise, so the printed bill is correct to the paisa.
- Units after roll-over correction lie between 0 and 65,535.

1.5 Assumptions

- One reading pair per household per month, and the meter rolls over at most once between two readings.
- A fall in reading is treated as a genuine roll-over only if previous >= 60,000 and current <= 1,000; any other fall is a meter fault or a typing error and is reported as invalid (used in Task 9).
- Slabs are progressive (each slab is charged at its own rate), the fixed charge is charged even for 0 units, and no taxes are added.
- Meter readings are whole units; one tariff applies to all categories for now.

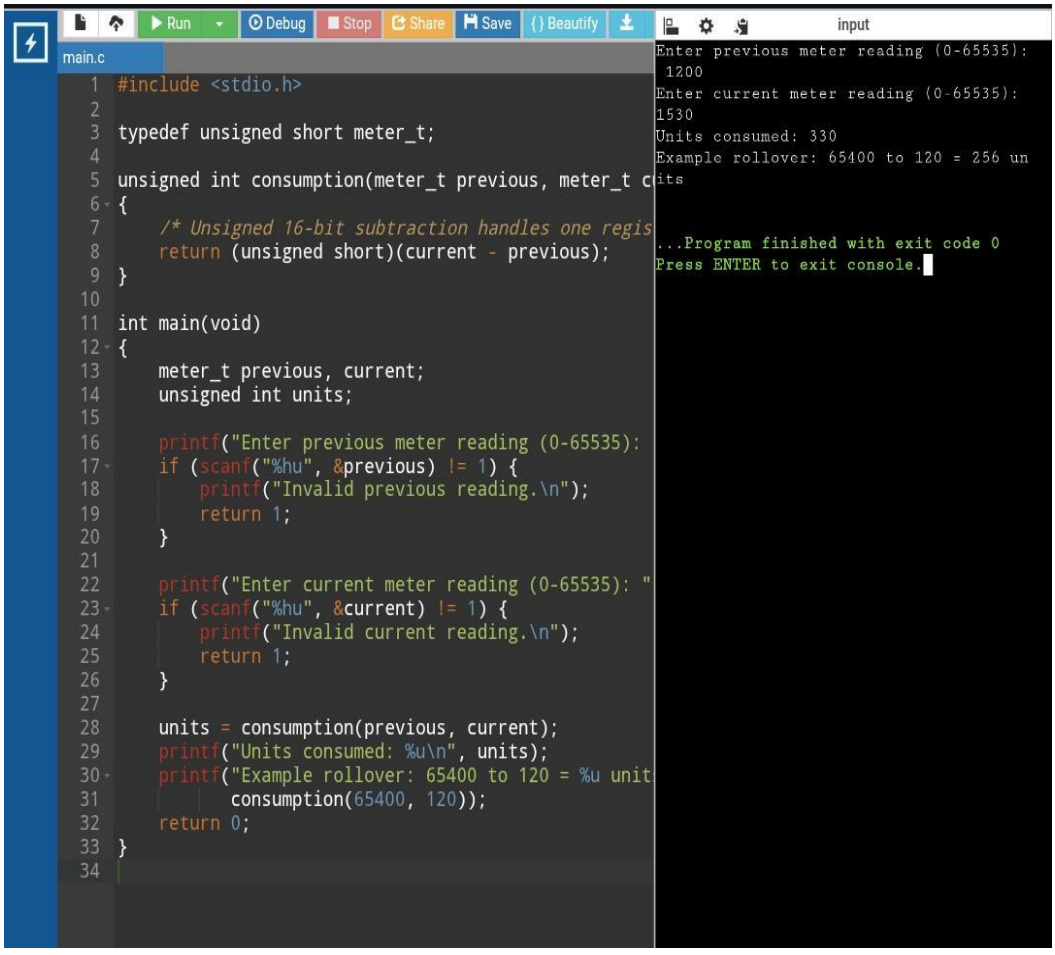
Alternative Concept and Complexity Analysis

If readings were stored as text such as "01530": the program would hold each reading in a char array of d + 1 bytes (d digits plus the terminating NUL), compare with strcmp, and convert with strtol or atoi before any subtraction. To subtract without converting, digit-by-digit subtraction with borrow would be needed, and roll-over detection becomes awkward.

	Integer (unsigned short)	Text (char[d+1])
Conversion text to number	once, on input: O(d)	repeated at every use: O(d)
Compare two readings	one machine instruction: O(1)	strcmp: O(d) (and correct only if zero-padded to equal length)
Subtract / roll-over	one unsigned subtraction: O(1)	convert both O(d), then subtract
Space per reading (d = 5)	2 bytes	6 bytes (3 times more)
Space for 800,000 houses x 2 readings	3.2 MB	9.6 MB

Since d is fixed at 5 the time is O(1) in both, but the text version has a larger constant factor and three times the memory. Text is suitable for reading and printing; integers are right for computing.

Evidence: Inputs, outputs, constants, rules and assumptions are identified above, with the reason for each type of storage.



TASK 2 - Choose the Data Types

2.1 Data type table

Sizes and ranges below come from the program in section 2.3 (sizeof, limits.h, float.h) run on gcc / Linux x86-64.

Data item	C type	sizeof	Range	Enough for a district?
household number	unsigned int	4 B	0 to 4,294,967,295	Yes (800,000 houses)
category	char	1 B	-128 to 127 (signed char)	Yes (a few codes)
previous / current reading	unsigned short (meter_t)	2 B	0 to 65,535	Yes - it matches the 16-bit register exactly
units consumed	unsigned int	4 B	0 to 4,294,967,295	Yes (at most 65,535 per bill)
bill, energy charge (paise)	long	8 B here	+/- 9,223,372,036,854,775,807	Yes for one bill. The C standard only guarantees 32 bits for long, so for portability use long long or int64_t.
district total (paise)	long long	8 B	+/- 9.22×10^{18}	Yes: 3.3×10^{10} is 277 million times smaller than the limit
(rejected) money as float	float	4 B	about 3.4×10^{38}, only 6 correct digits (FLT_DIG = 6)	No - rounding errors
(rejected) money as double	double	8 B	15 correct digits (DBL_DIG = 15)	Works, but a decimal fraction is still inexact

2.2 Why money should not be float rupees, and what happens after 65535

A value such as Rs 162.35 has no exact binary representation, so every float addition rounds. A float also keeps only about seven significant digits: once a total passes about Rs 500,000 the gap between neighbouring floats is 0.0625, so each added bill is rounded. Storing the amount as an integer number of paise (16235 for Rs 162.35) makes every addition exact. A 16-bit unsigned register holding 65535 wraps to 0 on the next increment, because unsigned arithmetic is performed modulo $2^{16} = 65,536$.

2.3 Program: sizes, limits, roll-over and the float failure

Source code - t2b.c

```
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main(void)
{
    printf("char=%zu short=%zu int=%zu long=%zu long long=%zu float=%zu double=%zu\n",
           sizeof(char), sizeof(short), sizeof(int), sizeof(long),
           sizeof(long long), sizeof(float), sizeof(double));
    printf("USHRT_MAX = %u\n", (unsigned)USHRT_MAX);
    printf("INT_MAX   = %d\n", INT_MAX);
    printf("UINT_MAX  = %u\n", UINT_MAX);
    printf("LONG_MAX  = %ld\n", LONG_MAX);
    printf("LLONG_MAX = %lld\n", LLONG_MAX);
    printf("FLT_DIG = %d, FLT_EPSILON = %g\n", FLT_DIG, (double)FLT_EPSILON);
    printf("DBL_DIG = %d, DBL_EPSILON = %g\n", DBL_DIG, DBL_EPSILON);

    /* 16-bit register rolling over */
    unsigned short reg = 65535;
    reg = reg + 1;
    printf("65535 + 1 in unsigned short = %u\n", (unsigned)reg);

    /* 4000 bills of Rs 162.35 : float rupees versus integer paise */
    float  total_f = 0.0f;
    long   total_p = 0;
    for (int i = 0; i < 4000; i++) {
        total_f += 162.35f;
        total_p += 16235L;
    }
    printf("float  total  = Rs %.2f\n", (double)total_f);
    printf("paise  total  = Rs %ld.%02ld\n", total_p / 100, total_p % 100);
    printf("difference    = Rs %.2f\n", (double)total_f - total_p / 100.0);
    return 0;
}
```

Output (terminal run, gcc 13.3, Linux x86-64) - output of t2b.c

```
$ gcc -Wall -Wextra t2b.c -o t2b
$ ./t2b
char=1 short=2 int=4 long=8 long long=8 float=4 double=8
USHRT_MAX = 65535
INT_MAX   = 2147483647
UINT_MAX  = 4294967295
LONG_MAX  = 9223372036854775807
LLONG_MAX = 9223372036854775807
FLT_DIG = 6, FLT_EPSILON = 1.19209e-07
DBL_DIG = 15, DBL_EPSILON = 2.22045e-16
65535 + 1 in unsigned short = 0
float  total  = Rs 649405.00
paise  total  = Rs 649400.00
difference    = Rs 5.00
```

Evidence: Adding 4,000 bills of Rs 162.35: float gives Rs 649,405.00, but the exact answer is Rs 649,400.00 - an error of Rs 5.00. The integer-paise total is exactly Rs 649,400.00, so the paise design avoids the problem.

2.4 Complete the Code - TODO 1 (no if-else)

The statement used is `return (unsigned short)(current - previous);`

Source code - t2a.c (TODO 1 completed)

```c
#include <stdio.h>
#include <limits.h>

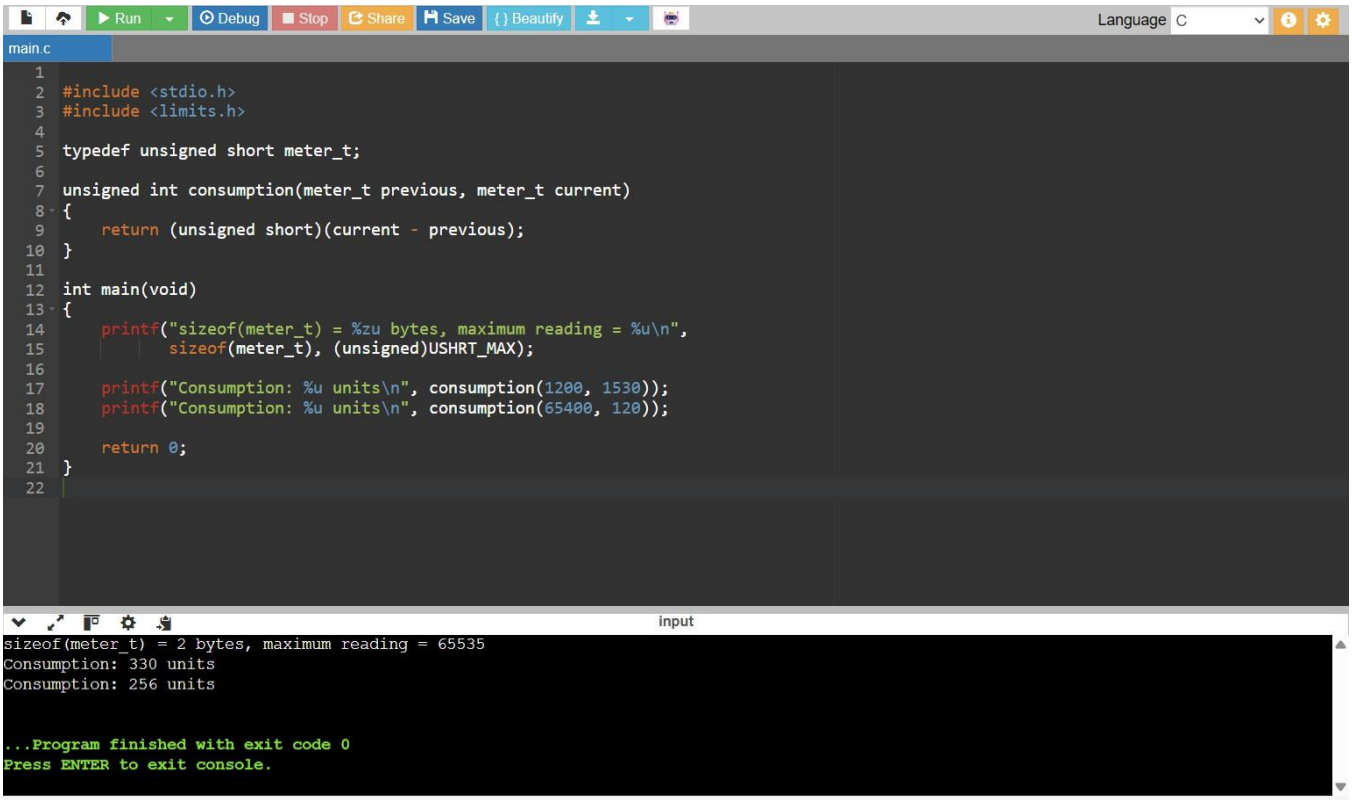
/* The meter register is a 16-bit counter: after 65535 it rolls over to 0. */
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    return (unsigned short)(current - previous);
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);

    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120)); /* rolled over */

    return 0;
}
```



Output screenshot - Task 2 (OnlineGDB): sizeof(meter_t) = 2, Consumption 330 and 256.

Why the second result is 256 and not negative. Both unsigned short operands are promoted to int, so current - previous = 120 - 65400 = -65280. Casting that value back to unsigned short reduces it modulo 65,536: -65280 + 65536 = 256. This equals the real consumption: 136 units from 65400 to the roll-over at 65536, plus 120 units after it = 256.

Alternative Concept and Complexity Analysis

If money were double rupees: the bill would be 50.00 + 2.25 x units-in-slab-2 + 4.50 x units-in-slab-3 computed in double. Rounding would have to be applied explicitly, once per bill, to the nearest paisa (for example llround(bill * 100) / 100.0), and again whenever a printed total must be cross-checked; without it two bills that look equal may differ in the last bits.

	long paise	double rupees
Time per addition	O(1), exact integer add	O(1), floating-point add (slightly slower)
Space per value	8 bytes	8 bytes (float would be 4 bytes)
Error after n additions	zero	each add may be off by up to 1.1×10^{-16} relative; worst case grows about linearly in n, typical case about sqrt(n)
Measured (Task 10)	0.00	double: below 10^{-8} rupees; float: Rs 282,918.50 at 800,000 bills

Evidence: A justified type table with sizes and ranges (2.1) and one precision problem (float total off by Rs 5.00) with its resolution (integer paise).

TASK 3 - Define the Constants

3.1 #define, const and enum

	#define	const variable	enum
What it is	Text replaced by the preprocessor before compilation	A real variable that cannot be modified	A set of named integer constants
Type checking	None (no type)	Yes (type is checked)	Integers (int) only
Scope	From the #define line to end of file (or #undef); ignores blocks	Normal block / file scope	Normal scope rules
Debugger	Name is gone - the debugger sees only the number	Visible and can be printed	Names are visible
Storage	None	May occupy memory (often optimised away)	None
Best use here	Tariff rates, slab limits, fixed charge	Typed constants such as a double conversion factor	Related codes such as category: CAT_DOMESTIC, CAT_COMMERCIAL

Naming conventions: UPPER_CASE_WITH_UNDERSCORES for macros and constants; put the unit in the name (RATE2_PAISE, SLAB2_UNITS); group related names with a common prefix (SLAB1_, RATE1_); no "magic numbers" inside the code; enum constants share a prefix (CAT_).

3.2 Complete the Code - TODO 1

Source code - t3.c

```
#include <stdio.h>

#define SLAB1_UNITS         100   /* first 100 units */
#define SLAB2_UNITS         200   /* units 101 to 200 */
#define RATE1_PAISE           0   /* first slab is free */
#define RATE2_PAISE         225   /* Rs 2.25 per unit */
#define RATE3_PAISE         450   /* Rs 4.50 per unit above 200 */
#define FIXED_CHARGE_PAISE 5000L /* Rs 50 fixed charge */

/* Returns the bill in paise, so that no rounding error can occur. */
long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB1_UNITS)
    {
        int second = units > SLAB2_UNITS ?
                     SLAB2_UNITS - SLAB1_UNITS :
                     units - SLAB1_UNITS;

        paise += second * RATE2_PAISE;
    }

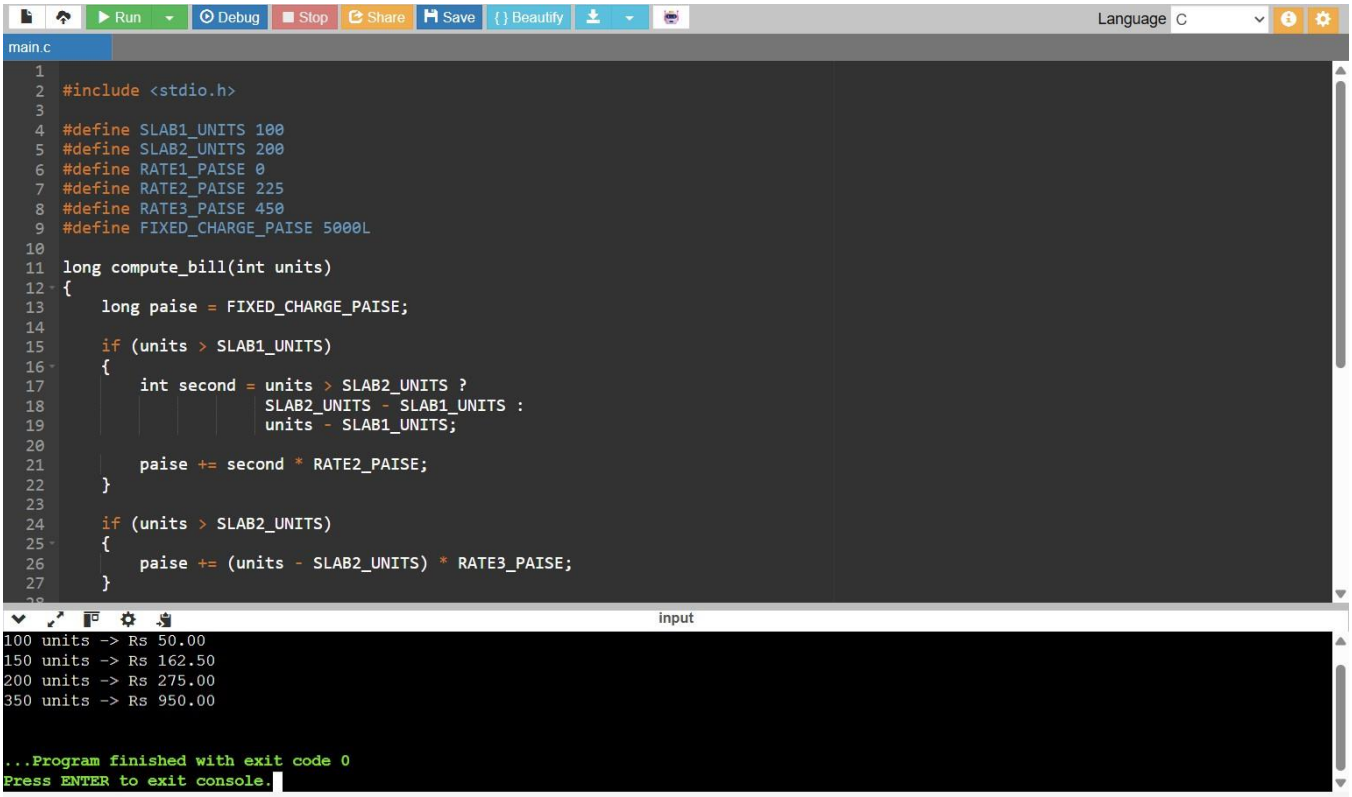
    if (units > SLAB2_UNITS)
    {
        paise += (units - SLAB2_UNITS) * RATE3_PAISE;
    }

    return paise;
}

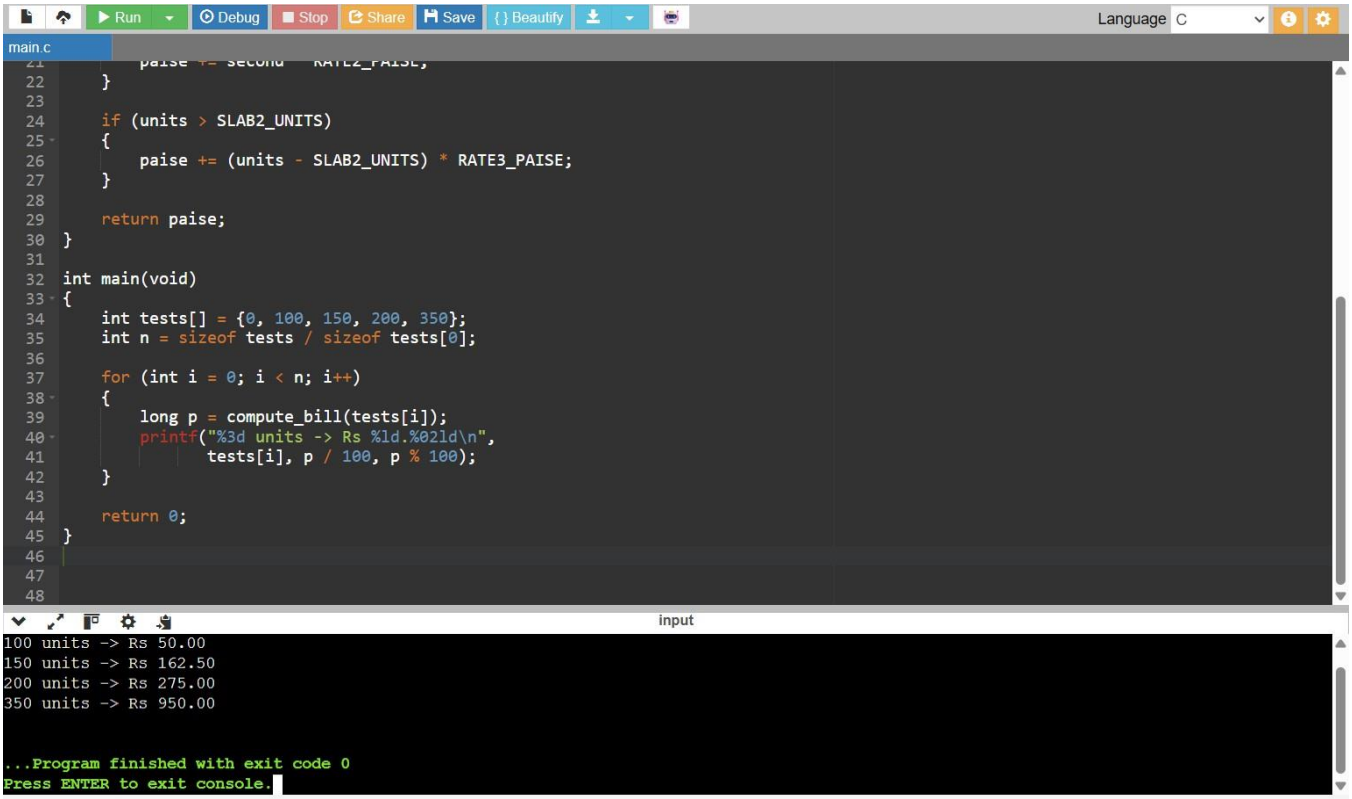
int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];

    for (int i = 0; i < n; i++)
    {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n", tests[i], p / 100, p % 100);
    }

    return 0;
}
```



Output screenshot - Task 3 (OnlineGDB): macros and the first part of compute_bill().



Output screenshot - Task 3 (OnlineGDB): rest of compute_bill(), main() and the expected output 50.00, 162.50, 275.00, 950.00.

3.3 Change RATE2_PAISE to 230

Only the single line `#define RATE2_PAISE 230` was edited. Bills with no second-slab units (0 and 100 units) stay at Rs 50.00; every other bill rises consistently: 150 units: 50 + 50 x 2.30 = Rs 165.00; 200 units: 50 + 100 x 2.30 = Rs 280.00; 350 units: 50 + 230 + 150 x 4.50 = Rs 955.00 (the slab-3 part is unchanged).

Output (terminal run, gcc 13.3, Linux x86-64) - tariff revision to 230 paise

```
$ grep RATE2_PAISE t3_230.c | head -1
#define RATE2_PAISE        230   /* Rs 2.30 per unit */
$ gcc -Wall -Wextra t3_230.c -o t3_230
$ ./t3_230
   0 units -> Rs 50.00
 100 units -> Rs 50.00
 150 units -> Rs 165.00
 200 units -> Rs 280.00
 350 units -> Rs 955.00
```

Alternative Concept and Complexity Analysis

With const variables: `const long RATE2_PAISE = 225;` keeps the same logic but adds type checking and the name appears in the debugger. It may take 8 bytes of read-only memory (a #define takes none). In C a const int is not a constant expression, so it cannot be used as an array size or a case label in older standards. **With an enum:** `enum { RATE2_PAISE = 225 };` needs no storage and is visible in the debugger, but holds only int values and groups best with related codes. **Running time:** not affected - with optimisation the compiler folds const values into the instructions, exactly as the preprocessor does for #define.

Evidence: One well-named set of constants (SLABI_UNITS ... FIXED_CHARGE_PAISE); #define chosen for tariff values because it costs no storage and can be used in preprocessor conditions, enum for category codes.

TASK 4 - Use Macros Safely

4.1 Macros

Source code - t4.c

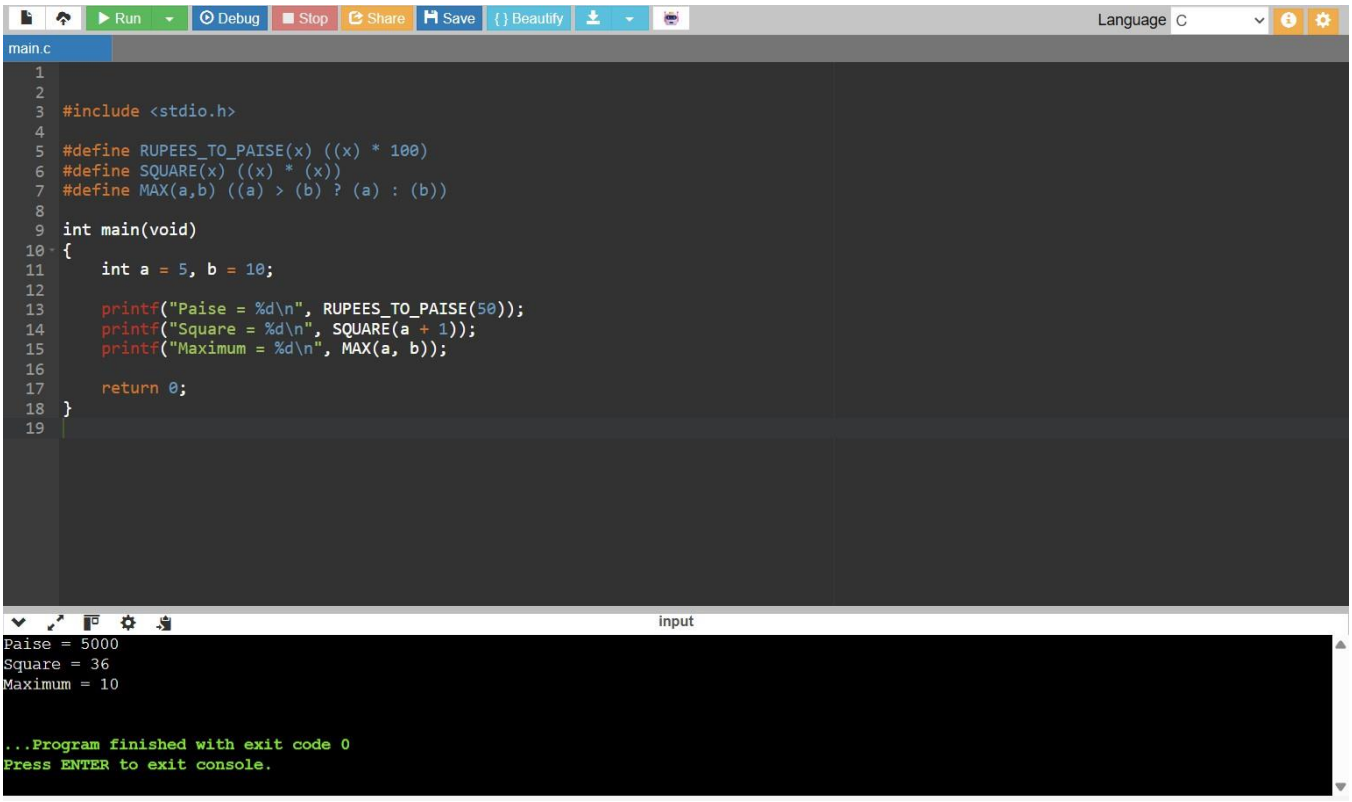
```
#include <stdio.h>

#define RUPEES_TO_PAISE(x) ((x) * 100)
#define SQUARE(x) ((x) * (x))
#define MAX(a,b) ((a) > (b) ? (a) : (b))

int main(void)
{
    int a = 5, b = 10;

    printf("Paise = %d\n", RUPEES_TO_PAISE(50));
    printf("Square = %d\n", SQUARE(a + 1));
    printf("Maximum = %d\n", MAX(a, b));

    return 0;
}
```



Output screenshot - Task 4 (OnlineGDB): Paise = 5000, Square = 36, Maximum = 10.

4.2 How the preprocessor expands them (gcc -E)

The preprocessor replaces each macro call with its text before compilation starts. The command `gcc -E t4.c` shows the result (last lines of the output):

Output (terminal run, gcc 13.3, Linux x86-64) - gcc -E evidence

```
$ gcc -E t4.c | tail -9
{
    int a = 5, b = 10;
    printf("Paise = %d\n", ((50) * 100));
    printf("Square = %d\n", ((a + 1) * (a + 1)));
    printf("Maximum = %d\n", ((a) > (b) ? (a) : (b)));
    return 0;
}
```

4.3 Pitfalls demonstrated

Source code - t4b.c

```c
#include <stdio.h>

#define BAD_SQUARE(x) x * x            /* no parentheses           */
#define SQUARE(x)     ((x) * (x))      /* fully parenthesised      */
#define MAX(a,b)      ((a) > (b) ? (a) : (b))

static inline int max_fn(int a, int b) { return a > b ? a : b; }

int main(void)
{
    int a = 5;
    printf("BAD_SQUARE(a + 1) = %d   (expected 36)\n", BAD_SQUARE(a + 1));
    printf("SQUARE(a + 1)     = %d\n", SQUARE(a + 1));

    int i = 5, j = 3;
    int m = MAX(i++, j);            /* i++ is evaluated twice */
    printf("MAX(i++, j)       = %d, i is now %d  (expected m=5, i=6)\n", m, i);

    i = 5;
    m = max_fn(i++, j);             /* function: argument evaluated once */
    printf("max_fn(i++, j)    = %d, i is now %d\n", m, i);
    return 0;
}
```

Output (terminal run, gcc 13.3, Linux x86-64) - pitfalls

```
$ gcc -Wall -Wextra t4b.c -o t4b
$ ./t4b
BAD_SQUARE(a + 1) = 11   (expected 36)
SQUARE(a + 1)     = 36
MAX(i++, j)       = 6, i is now 7  (expected m=5, i=6)
max_fn(i++, j)    = 5, i is now 6
```

- **SQUARE(x) written as x*x:** SQUARE(a+1) expands to a+1*a+1 = 5 + 5 + 1 = 11, not 36, because * binds tighter than +. Parenthesising every use of the argument and the whole body, ((x) * (x)), gives 36.
- **MAX evaluates an argument twice:** MAX(i++, j) expands to ((i++) > (j) ? (i++) : (j)). With i = 5 and j = 3 the test increments i to 6, the second i++ returns 6 (not 5) and i ends at 7. A function evaluates its arguments once, giving 5 and i = 6.
- **A function is better** when arguments may have side effects, when the body is long (a macro is copied at every use), when type checking is needed, when the address or recursion is needed, or when the code must be easy to debug (a macro cannot be stepped through).

Alternative Concept and Complexity Analysis

With static inline functions: `static inline int max_fn(int a, int b) { return a > b ? a : b; }` and a similar square function. Arguments are evaluated once, the types are checked, and the function can be stepped through in a debugger.

	Macro	static inline function
Time per use	No call: expression in place	Same when the compiler inlines it (-O1/-O2); otherwise one call and return (a few nanoseconds)
Code size for k uses	k copies of the expression	One copy plus k call instructions if not inlined; k copies if inlined
Type safety	None	Yes (but it works for int only - a macro works for any numeric type)

Evidence: Fully parenthesised macros, gcc -E output and an explanation of each pitfall (missing parentheses and double evaluation).

TASK 5 - Organise the Program with Header Files and Conditional Compilation

5.1 Structure of the program

A C program is arranged as: (1) #include lines for header files, (2) global declarations - constants, typedefs and function prototypes, (3) function definitions, and (4) main(). The tariff constants, the DEBUG switch and the prototype are placed in **billing.h**; compute_bill() is defined in **billing.c**; **main5.c** contains main().

- **Include guards** (#ifndef BILLING_H / #define BILLING_H / #endif): if a header is included twice, directly or through another header, its typedefs and prototypes would be declared twice and compilation would fail. The guard makes the second inclusion empty.
- **#include <file>** searches the system include directories (standard library headers); **#include "file"** searches the current directory first, then the system directories - used for our own billing.h.
- **#ifdef DEBUG** keeps the intermediate print statements only if the program is compiled with gcc -DDEBUG; otherwise they are removed before compilation.

Source code - billing.h

```
#ifndef BILLING_H
#define BILLING_H

/* ---- tariff constants : the ONLY place a tariff revision is made ---- */
#define SLAB1_UNITS         100
#define SLAB2_UNITS         200
#define RATE1_PAISE           0
#define RATE2_PAISE         225
#define RATE3_PAISE         450
#define FIXED_CHARGE_PAISE 5000L

/* ---- debugging switch: compile with  gcc -DDEBUG ... ---- */
#ifdef DEBUG
    #define DBG(...) printf(__VA_ARGS__)
#else
    #define DBG(...) ((void)0)
#endif

/* ---- prototype ---- */
long compute_bill(int units);

#endif /* BILLING_H */
```

Source code - billing.c

```
#include <stdio.h>
#include "billing.h"

long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;
    DBG("[DEBUG] units = %d, fixed = %ld\n", units, paise);

    if (units > SLAB1_UNITS)
    {
        int second = units > SLAB2_UNITS ? SLAB2_UNITS - SLAB1_UNITS
                                         : units - SLAB1_UNITS;
        paise += second * RATE2_PAISE;
        DBG("[DEBUG] slab 2 units = %d, bill so far = %ld\n", second, paise);
    }
    if (units > SLAB2_UNITS)
    {
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
        DBG("[DEBUG] slab 3 units = %d, bill so far = %ld\n",
            units - SLAB2_UNITS, paise);
    }
    return paise;
}
```

Source code - main5.c

```
#include <stdio.h>
#include "billing.h"

int main(void)
{
    int units = 350;
    long p = compute_bill(units);
    printf("%d units -> Rs %ld.%02ld\n", units, p / 100, p % 100);
    return 0;
}
```

Output (terminal run, gcc 13.3, Linux x86-64) - normal build, then gcc -DDEBUG build

```
$ gcc -Wall -Wextra billing.c main5.c -o bill_release
$ ./bill_release
350 units -> Rs 950.00
$ gcc -Wall -Wextra -DDEBUG billing.c main5.c -o bill_debug
$ ./bill_debug
[DEBUG] units = 350, fixed = 5000
[DEBUG] slab 2 units = 100, bill so far = 27500
[DEBUG] slab 3 units = 150, bill so far = 95000
350 units -> Rs 950.00
$ ls -l bill_release bill_debug | awk '{print $5, $9}'
16032 bill_debug
16032 bill_release
```

Alternative Concept and Complexity Analysis

With a run-time variable (`int debug = 0; ... if (debug) printf(...);`) the debug code is always compiled in. Every call pays one extra test and branch, O(1) and very small, but not zero; the debug strings and code remain in the executable, and debugging can be turned on without recompiling. With #ifdef the release program has no test and no debug code. Measured on billing.c: the object file has 183 bytes of code without -DDEBUG and 417 bytes with it (command: size).

Output (terminal run, gcc 13.3, Linux x86-64) - size of billing.o without and with -DDEBUG (r.o = release, d.o = debug)

```
   text□   data□    bss□    dec□    hex□filename
    183□      0□      0□    183□     b7□r.o
    417□      0□      0□    417□    1a1□d.o
```

Evidence: A header with include guards, a working DEBUG mode (the [DEBUG] lines appear only with -DDEBUG) and an explanation of the structure.

TASK 6 - Read and Print Data Correctly

6.1 Format specifiers

Type	scanf	printf	Used for
int	%d	%d	readings read as int for range checking
unsigned int	%u	%u	units consumed
long	%ld	%ld	paise amounts
long long	%lld	%lld	district totals
float / double	%f / %lf	%f / %f	not used for money (note: scanf needs %lf for double)
char / string	%c / %s	%c / %s	category code / names

6.2 A wrong specifier

Source code - t6b.c

```c
#include <stdio.h>

int main(void)
{
    long big = 5000000000L;          /* Rs 5 crore in paise */
    printf("correct %%ld : %ld\n", big);
    printf("wrong   %%d  : %d\n", (int)big);   /* what %d would show: low 32 bits */
    printf("%%d with a long argument (undefined behaviour): %d\n", big);
    printf("%%f with an int argument (undefined behaviour): %f\n", 5);
    return 0;
}
```

Output (terminal run, gcc 13.3, Linux x86-64) - the first line is the number of compiler warnings; the program then prints wrong values

```
$ gcc t6b.c -o t6b 2>&1 | grep -c warning
2
$ ./t6b
correct %ld : 5000000000
wrong   %d  : 705032704
%d with a long argument (undefined behaviour): 705032704
%f with an int argument (undefined behaviour): 0.000000
```

Printing a long with %d shows only the low 32 bits: 5,000,000,000 becomes 705,032,704. The behaviour is undefined, the compiler warns (-Wformat), and the bill would be silently wrong.

6.3 Reading a bill amount with scanf and checking its return value

Source code - t6a.c

```c
#include <stdio.h>

int main(void)
{
    long paise;

    printf("Enter bill amount in paise: ");

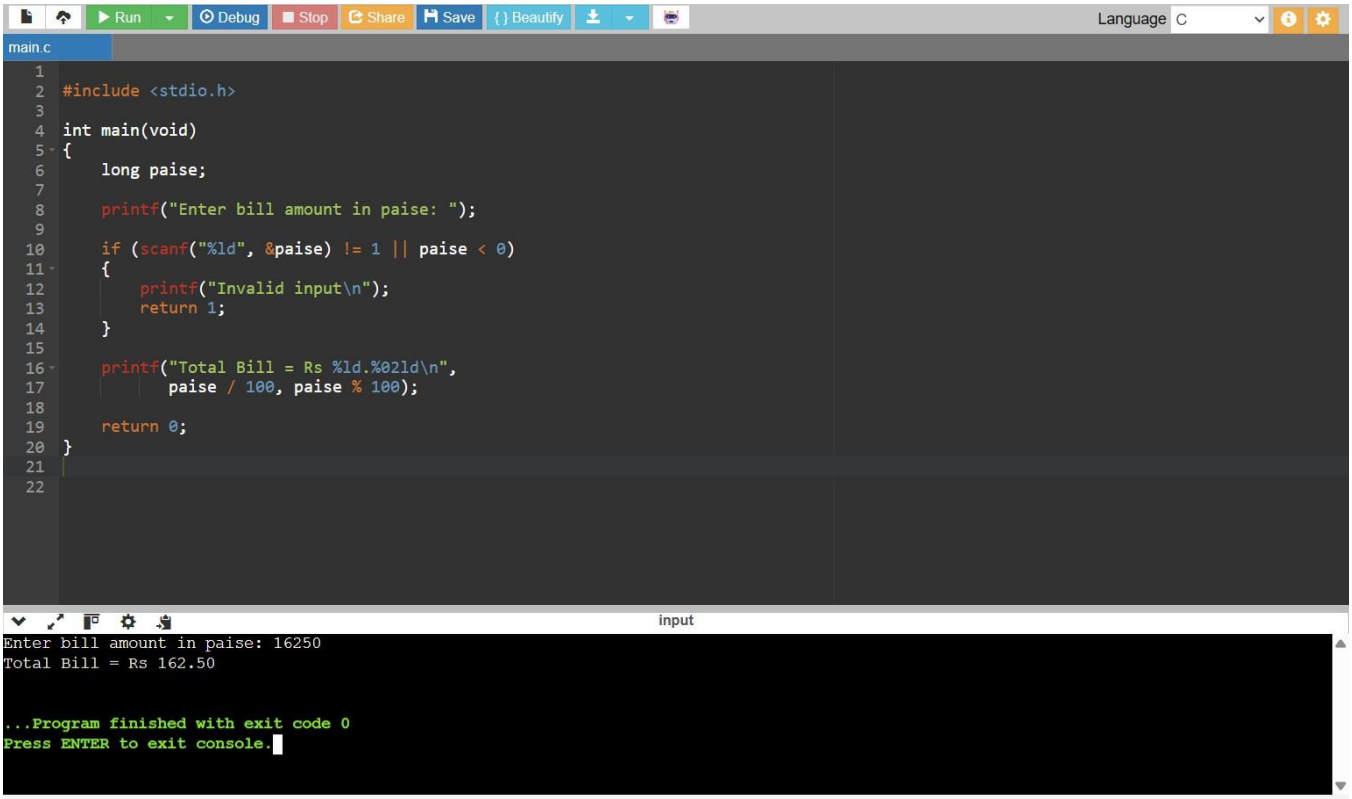
    if (scanf("%ld", &paise) != 1 || paise < 0)
    {
        printf("Invalid input\n");
        return 1;
    }

    printf("Total Bill = Rs %ld.%02ld\n", paise / 100, paise % 100);

    return 0;
}
```



Screenshot - Task 6 (OnlineGDB): program waiting for input.



Output screenshot - Task 6 (OnlineGDB): input 16250 prints "Total Bill = Rs 162.50".

Output (terminal run, gcc 13.3, Linux x86-64) - invalid input (letters and a negative value) is rejected because scanf returns 0 or the value is negative

```
$ echo abc | ./t6a
Enter bill amount in paise: Invalid input
$ echo -5 | ./t6a
Enter bill amount in paise: Invalid input
```

scanf returns the number of items successfully read; comparing it with 1 detects letters. The amount in paise is printed as rupees and paise with paise / 100 and paise % 100 using %02ld, so 16250 prints as 162.50 and 16205 as 162.05.

Alternative Concept and Complexity Analysis

With fgets() and strtol(): the whole line is read into a buffer, then converted.

Source code - t6c.c

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    char input[100];
    char *end;

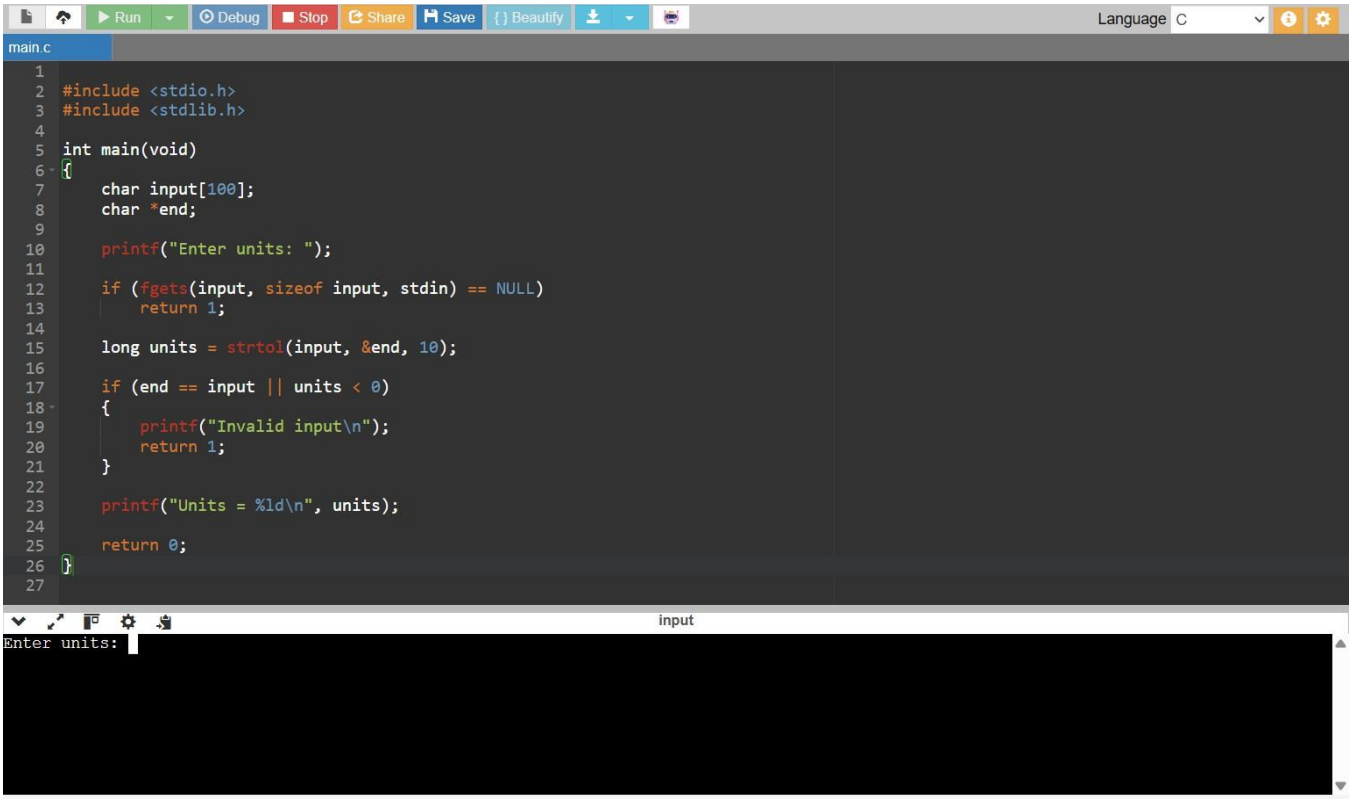
    printf("Enter units: ");

    if (fgets(input, sizeof input, stdin) == NULL)
        return 1;

    long units = strtol(input, &end, 10);

    if (end == input || units < 0)
    {
        printf("Invalid input\n");
        return 1;
    }
    printf("Units = %ld\n", units);

    return 0;
}
```



Screenshot - Task 6 alternative (OnlineGDB): fgets + strtol program waiting for input.

Output (terminal run, gcc 13.3, Linux x86-64) - valid, invalid and partly valid input

```
$ gcc -Wall -Wextra t6c.c -o t6c
$ echo 1530 | ./t6c
Enter units: Units = 1530
$ echo abc | ./t6c
Enter units: Invalid input
$ echo 12abc | ./t6c
Enter units: Units = 12
```

- **Time:** fgets copies the line, O(L); strtol scans the digits, O(L); scanf also reads O(L) characters.
- **Space:** fgets needs a buffer of L + 1 bytes (100 bytes here); scanf("%ld") needs no buffer of its own.
- **Recovery from invalid input:** after a failed scanf the bad characters stay in the input buffer, so the next scanf fails again unless the buffer is cleared. With fgets the full line is already consumed, so the program

can print an error and continue with the next line. strtol also tells exactly where conversion stopped through its end pointer.

- **Limitation seen in the output:** "12abc" is accepted as 12. The fix is to also require that *end is the newline or the end of the string.

Evidence: Correct format specifiers, input validation using the return value of scanf, and a correctly formatted bill (Rs 162.50).

TASK 7 - Control Type Conversion and Precision

7.1 Where different types are combined in the billing program

Place in the program	Conversion that happens
current - previous (both unsigned short)	Both are promoted to int; the result is converted back to unsigned short, which gives the roll-over correction
units = current - previous with int readings	int result stored in unsigned int; safe only after checking current >= previous
second * RATE2 and (units - 200) * RATE3	int times int is computed in int; the product is added to a long. The cast (long) before multiplying prevents int overflow for large units
paise / 100 and paise % 100 when printing	Intentional integer division: rupees and paise are separated exactly
units / 100 (if rupees were wanted)	Integer division discards the fraction - wrong for money
scanf("%d") compared with 65535	int compared with int; a value above INT_MAX would already have overflowed on reading

7.2 Surprising results

Source code - t7.c

```c
#include <stdio.h>

int main(void)
{
    int  units = 250;
    long big   = 5000000000L;
    double x   = 1.15;

    /* 1. integer division */
    printf("1) units / 100         = %d      (rupees lost: 250/100 is 2, not 2.5)\n", units / 100);
    printf("   units / 100.0       = %.2f\n", units / 100.0);

    /* 2. narrowing cast of a large long */
    printf("2) (int)5000000000L    = %d\n", (int)big);

    /* 3. usual arithmetic conversions: signed -> unsigned */
    int neg = -1;
    unsigned int one = 1;
    printf("3) (-1 < 1u)           = %d      (expected 1)\n", neg < one);

    /* 4. truncation instead of rounding */
    printf("4) (long)(1.15*100)    = %ld   (expected 115)\n", (long)(x * 100));
    printf("   (long)(1.15*100 + 0.5) = %ld\n", (long)(x * 100 + 0.5));

    /* 5. integer promotion on a 16-bit register */
    unsigned short r = 65535;
    printf("5) r + 1 (promoted)    = %d, but (unsigned short)(r+1) = %u\n",
           r + 1, (unsigned)(unsigned short)(r + 1));
    return 0;
}
```

Output (terminal run, gcc 13.3, Linux x86-64) - output of t7.c (first line: the compiler warning about case 3)

```
$ gcc -Wall -Wextra t7.c -o t7 2>&1 | grep warning | head -3
t7.c:19:67: warning: comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign-compare]
$ ./t7
1) units / 100         = 2      (rupees lost: 250/100 is 2, not 2.5)
   units / 100.0       = 2.50
2) (int)5000000000L    = 705032704
3) (-1 < 1u)           = 0      (expected 1)
4) (long)(1.15*100)    = 114   (expected 115)
   (long)(1.15*100 + 0.5) = 115
5) r + 1 (promoted)    = 65536, but (unsigned short)(r+1) = 0
```

- **1) units / 100 = 2:** both operands are int, so the division is integer division and 0.50 is lost. Use units / 100.0 or keep paise.
- **2) (int)5000000000L = 705032704:** the cast keeps only the low 32 bits of a long; the value silently changes. Never cast an amount to a smaller type.
- **3) (-1 < 1u) is 0 (false):** in the usual arithmetic conversions the int -1 is converted to unsigned int and becomes 4,294,967,295. Mixing signed and unsigned in a comparison is the reason units must be checked before subtracting.

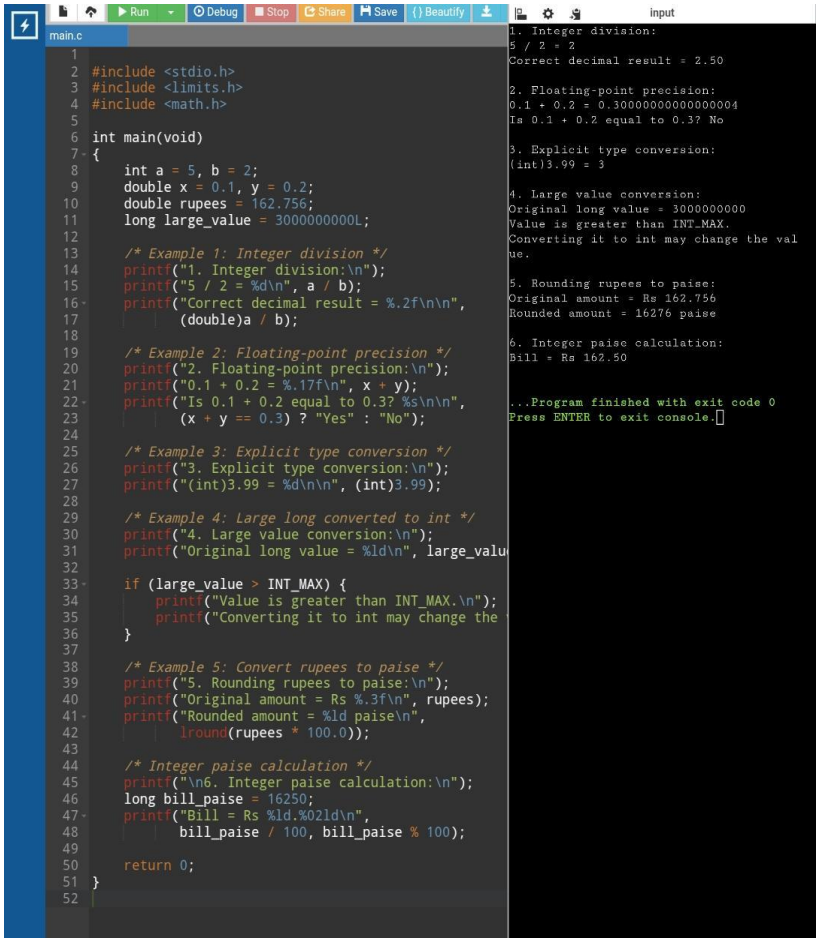
- **4) (long)(1.15*100) = 114:** 1.15 is stored as 1.149999..., so 1.15 x 100 is 114.99999999999999 and a cast truncates toward zero. Adding 0.5 before the cast rounds to 115.
- **5) r + 1 = 65536 but (unsigned short)(r + 1) = 0:** integer promotion makes r + 1 an int, so it does not wrap; the cast back to 16 bits does.

Rounding rule used: all money is computed in integer paise. The rates are whole paise, so no fraction can arise in the original design. If a fractional paisa can occur (Task 12), accumulate in a finer integer unit and round half up once per bill: (x + half) / scale.

Alternative Concept and Complexity Analysis

If everything were calculated in float and rounded once at the end: the total of n bills is accumulated as float rupees and converted to paise at the end. The measured error (Task 10, same generated data in both methods) was: 1,000 bills Rs 0.00; 4,000 bills Rs 0.00; 60,000 bills Rs -49.25; 200,000 bills Rs -31,989.75; 800,000 bills Rs +282,918.50. The error grows quickly because a float has 24 bits of precision: when the total reaches millions, adding a bill of a few hundred rupees is rounded to a multiple of 2 or 4. Time per value: float add O(1), integer add O(1); space: float 4 bytes, long 8 bytes - the extra 4 bytes buys exact results.

Evidence: Conversion points identified, five surprises explained with output, and a consistent rounding rule (integer paise, one half-up rounding).



TASK 8 - Trace the Compilation Process

The macro program of Task 4 (t4.c, 16 lines) is followed through each stage.

8.1 Preprocessing: gcc -E

Output (terminal run, gcc 13.3, Linux x86-64) - preprocessed file t4.i

```
$ gcc -E t4.c -o t4.i && wc -l t4.c t4.i
   16 t4.c
  829 t4.i
  845 total
$ grep -n 'printf' t4.i | tail -3
824:    printf("Paise = %d\n", ((50) * 100));
825:    printf("Square = %d\n", ((a + 1) * (a + 1)));
826:    printf("Maximum = %d\n", ((a) > (b) ? (a) : (b)));
```

The preprocessor removed the comments and the #define lines, replaced RUPEES_TO_PAISE, SQUARE and MAX by their text, and replaced #include <stdio.h> by the declarations in that header, so 16 lines became 829 lines.

8.2 Compilation to assembly: gcc -S

Output (terminal run, gcc 13.3, Linux x86-64) - assembly file t4.s (first lines)

```
$ gcc -S t4.c -o t4.s && wc -l t4.s
73 t4.s
$ head -12 t4.s
□.file□"t4.c"
□.text
□.section□.rodata
.LC0:
□.string□"Paise = %d\n"
.LC1:
□.string□"Square = %d\n"
.LC2:
□.string□"Maximum = %d\n"
□.text
□.globl□main
□.type□main, @function
```

8.3 Object file and linking: gcc -c, then link

Output (terminal run, gcc 13.3, Linux x86-64) - object file (machine code, calls to printf not yet resolved), then the linked executable

```
$ gcc -c t4.c -o t4.o && ls -l t4.o | awk '{print $5, $9}'
1720 t4.o
$ gcc t4.o -o t4_exe && ls -l t4_exe | awk '{print $5, $9}'
15960 t4_exe
$ ./t4_exe
Paise = 5000
Square = 36
Maximum = 10
```

The object file t4.o (1,720 bytes) contains machine code with the address of printf left open; the linker joins it with the C library to create the executable (15,960 bytes).

8.4 The three kinds of error

Source code - the three small test programs (ce.c, le.c, re.c)

```
/* compile-time: ce.c */  int a = 5   <- missing semicolon
/* link-time   : le.c */  compute_bill() is called but never defined
/* run-time    : re.c */  total / units  with units = 0
```

Output (terminal run, gcc 13.3, Linux x86-64) - compile-time error, link-time error, run-time error

```
$ gcc ce.c -o ce
ce.c: In function 'main':
ce.c:5:5: error: expected ',' or ';' before 'printf'
    5 |     printf("%d\n", a);
      |     ^~~~~~
$ gcc le.c -o le
/usr/bin/ld: /tmp/ccLH1XRd.o: in function `main':
le.c:(.text+0xe): undefined reference to `compute_bill'
collect2: error: ld returned 1 exit status
$ gcc re.c -o re && ./re; echo exit code=$?
Floating point exception
exit code=136
```

- **Compile-time error:** the compiler cannot understand the source (syntax error); no object file is produced.
- **Link-time error:** every file compiled, but the linker found a call to a function with no definition ("undefined reference").
- **Run-time error:** the program built successfully but crashed while running (division by zero, signal "Floating point exception", exit code 136).

8.5 Warnings from gcc -Wall -Wextra

Output (terminal run, gcc 13.3, Linux x86-64) - warnings (t8err.c)

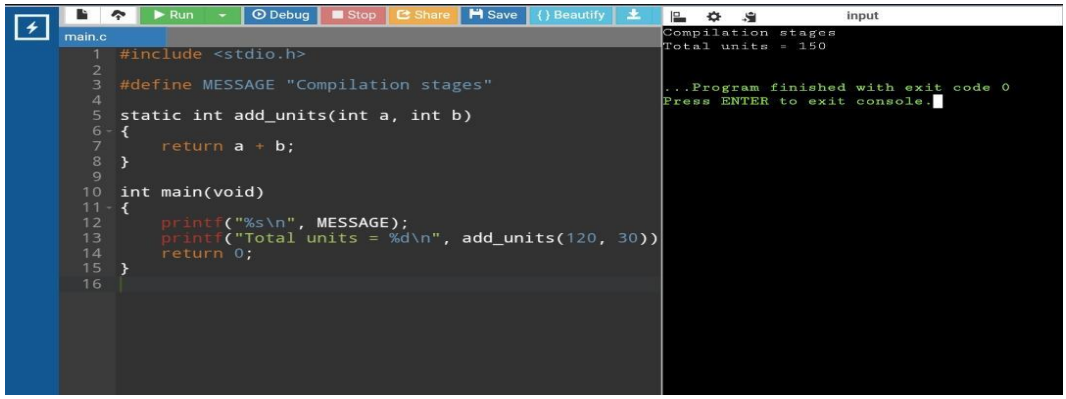
```
$ gcc -Wall -Wextra t8err.c -o t8err
t8err.c: In function 'main':
t8err.c:7:14: warning: format '%d' expects argument of type 'int', but argument 2 has type 'long int' [-Wformat=]
    7 |     printf("%d\n", p);          /* wrong specifier  -> warning */
      |             ~^     ~
      |              |     |
      |              int   long int
      |             %ld
t8err.c:8:12: warning: comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign-compare]
    8 |     if (-1 < u) puts("x");      /* sign compare     -> warning */
      |            ^
t8err.c:5:9: warning: unused variable 'unused' [-Wunused-variable]
    5 |     int unused;
      |         ^~~~~~
```

- **-Wformat:** %d used for a long. Resolved by using %ld.
- **-Wsign-compare:** comparing int with unsigned int. Resolved by comparing values of the same signedness or checking the sign first.
- **-Wunused-variable:** a variable never used. Resolved by deleting it.

Alternative Concept and Complexity Analysis

With several files (input.c, tariff.c, report.c) and a shared header: each file is compiled separately with `gcc -c file.c`, then `gcc input.o tariff.o report.o -o billing` links them (a Makefile automates this). After changing one file only that file is recompiled and the program is relinked; with a single file the entire program is recompiled every time, so compile time is proportional to the total size instead of the size of the changed file. The object files use somewhat more space in total (each has its own headers and symbol table) but are reusable; for a program as small as ours the difference is a few kilobytes and the single file is simpler.

Evidence: Output of every compilation stage, three correctly classified errors, and the meaning of each warning with its fix.



TASK 9 - Implement and Test the Billing Program

9.1 Complete source code

Source code - t9.c - complete billing program

```
#include <stdio.h>

#define RATE1 0
#define RATE2 225
#define RATE3 450
#define FIXED_CHARGE 5000L

long compute_bill(unsigned int units)
{
    long bill = FIXED_CHARGE;

    if (units > 100)
    {
        int second = units > 200 ? 100 : units - 100;
        bill += second * RATE2;
    }

    if (units > 200)
        bill += (long)(units - 200) * RATE3;

    return bill;
}

int main(void)
{
    int previous, current;
    unsigned int units;
    long bill;

    printf("Enter previous reading: ");

    if (scanf("%d", &previous) != 1 ||
        previous < 0 || previous > 65535)
    {
        printf("Invalid previous reading\n");
        return 1;
    }

    printf("Enter current reading: ");

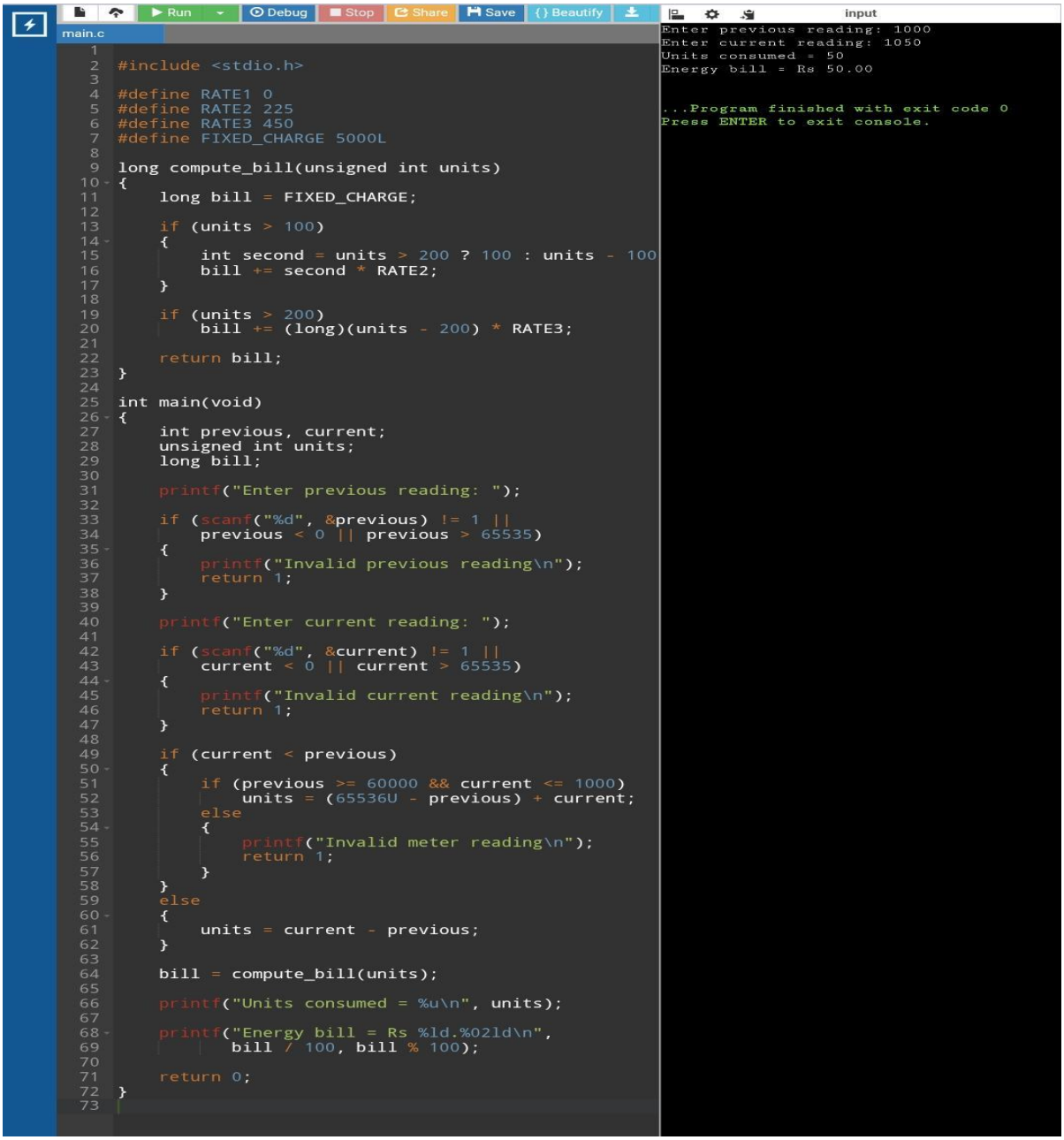
    if (scanf("%d", &current) != 1 ||
        current < 0 || current > 65535)
    {
        printf("Invalid current reading\n");
        return 1;
    }

    if (current < previous)
    {
        if (previous >= 60000 && current <= 1000)
            units = (65536U - previous) + current;
        else
        {
            printf("Invalid meter reading\n");
            return 1;
        }
    }
    else
    {
        units = current - previous;
    }

    bill = compute_bill(units);

    printf("Units consumed = %u\n", units);
    printf("Energy bill = Rs %ld.%02ld\n",
           bill / 100, bill % 100);

    return 0;
}
```




```c
#include <stdio.h>

#define RATE1 0
#define RATE2 225
#define RATE3 450
#define FIXED_CHARGE 5000L

long compute_bill(unsigned int units)
{
    long bill = FIXED_CHARGE;

    if (units > 100)
    {
        int second = units > 200 ? 100 : units - 100;
        bill += second * RATE2;
    }

    if (units > 200)
        bill += (long)(units - 200) * RATE3;

    return bill;
}

int main(void)
{
    int previous, current;
    unsigned int units;
    long bill;

    printf("Enter previous reading: ");

    if (scanf("%d", &previous) != 1 ||
        previous < 0 || previous > 65535)
    {
        printf("Invalid previous reading\n");
        return 1;
    }

    printf("Enter current reading: ");

    if (scanf("%d", &current) != 1 ||
        current < 0 || current > 65535)
    {
        printf("Invalid current reading\n");
        return 1;
    }

    if (current < previous)
    {
        if (previous >= 60000 && current <= 1000)
            units = (65536U - previous) + current;
        else
        {
            printf("Invalid meter reading\n");
            return 1;
        }
    }
    else
    {
        units = current - previous;
    }

    bill = compute_bill(units);

    printf("Units consumed = %u\n", units);

    printf("Energy bill = Rs %ld.%02ld\n",
           bill / 100, bill % 100);

    return 0;
}
```

Screenshot - Task 9 (OnlineGDB): units calculation, bill and printing, end of the program.

9.2 Test results

Each test was run with the previous reading on the first line and the current reading on the second line. Expected values were worked out by hand from the tariff (Rs 50 fixed; first 100 units free; next 100 at Rs 2.25; above 200 at Rs 4.50).

#	Case	Previous, current	Expected	Actual	Result
1	Slab 1	1000, 1050	50 units, Rs 50.00	50 units, Rs 50.00	Pass
2	Slab 2	1000, 1150	150 units, Rs 162.50	150 units, Rs 162.50	Pass
3	Slab 3	1000, 1350	350 units, Rs 950.00	350 units, Rs 950.00	Pass
4	Boundary: exactly 100 units	500, 600	Rs 50.00	Rs 50.00	Pass
5	Boundary: exactly 200 units	500, 700	Rs 275.00	Rs 275.00	Pass
6	Zero units	700, 700	0 units, Rs 50.00	0 units, Rs 50.00	Pass
7	Roll-over 65535 to 0	65400, 120	256 units, Rs 527.00 (50 + 225 + 56 x 4.50)	256 units, Rs 527.00	Pass
8	Invalid: letters	abc	Invalid previous reading	Invalid previous reading	Pass
9	Invalid: negative reading	-5	Invalid previous reading	Invalid previous reading	Pass
10	Invalid: current < previous, no roll-over	1500, 1200	Invalid meter reading	Invalid meter reading	Pass
11	Invalid: letters in current	100, xyz	Invalid current reading	Invalid current reading	Pass

Output (terminal run, gcc 13.3, Linux x86-64) - all 11 test runs

```
$ printf "1000\n1050\n" | ./t9
Enter previous reading: Enter current reading: Units consumed = 50
Energy bill = Rs 50.00
$ printf "1000\n1150\n" | ./t9
Enter previous reading: Enter current reading: Units consumed = 150
Energy bill = Rs 162.50
$ printf "1000\n1350\n" | ./t9
Enter previous reading: Enter current reading: Units consumed = 350
Energy bill = Rs 950.00
$ printf "500\n600\n" | ./t9
Enter previous reading: Enter current reading: Units consumed = 100
Energy bill = Rs 50.00
$ printf "500\n700\n" | ./t9
Enter previous reading: Enter current reading: Units consumed = 200
Energy bill = Rs 275.00
$ printf "700\n700\n" | ./t9
Enter previous reading: Enter current reading: Units consumed = 0
Energy bill = Rs 50.00
$ printf "65400\n120\n" | ./t9
Enter previous reading: Enter current reading: Units consumed = 256
Energy bill = Rs 527.00
$ printf "abc\n100\n" | ./t9
Enter previous reading: Invalid previous reading
$ printf "-5\n100\n" | ./t9
Enter previous reading: Invalid previous reading
$ printf "1500\n1200\n" | ./t9
Enter previous reading: Enter current reading: Invalid meter reading
$ printf "100\nxyz\n" | ./t9
Enter previous reading: Enter current reading: Invalid current reading
```

9.3 Five bills whose total is calculated by hand

House	Previous, current	Units	Hand calculation	Bill (hand)	Bill (program)
A	1000, 1040	40	50.00	Rs 50.00	Rs 50.00
B	1000, 1120	120	50 + 20 x 2.25 = 50 + 45	Rs 95.00	Rs 95.00
C	1000, 1180	180	50 + 80 x 2.25 = 50 + 180	Rs 230.00	Rs 230.00
D	1000, 1260	260	50 + 225 + 60 x 4.50 = 50 + 225 + 270	Rs 545.00	Rs 545.00
E	1000, 1410	410	50 + 225 + 210 x 4.50 = 50 + 225 + 945	Rs 1220.00	Rs 1220.00
Total			50 + 95 + 230 + 545 + 1220	**Rs 2140.00**	**Rs 2140.00**

Alternative Concept and Complexity Analysis

With slab limits and rates in two arrays and a loop over the slabs:

Source code - t9b.c

```c
#include <stdio.h>

#define NSLABS 3
static const long limit[NSLABS] = {100, 200, 1000000000L}; /* upper limit of slab  */
static const long rate [NSLABS] = {0, 225, 450};           /* paise per unit       */
#define FIXED 5000L

long compute_bill(long units)
{
    long bill = FIXED, lower = 0;
    for (int s = 0; s < NSLABS && units > lower; s++)
    {
        long top  = units < limit[s] ? units : limit[s];
        bill += (top - lower) * rate[s];
        lower = limit[s];
    }
    return bill;
}

int main(void)
{
    long tests[] = {0, 100, 150, 200, 350};
    for (int i = 0; i < 5; i++)
    {
        long p = compute_bill(tests[i]);
        printf("%3ld units -> Rs %ld.%02ld\n", tests[i], p / 100, p % 100);
    }
    return 0;
}
```

```
$ gcc -Wall -Wextra t9b.c -o t9b
$ ./t9b
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

- **Time per bill:** O(s) for s slabs (the loop), against a fixed number of tests (O(1)) for the if version. For s = 3 the difference is unmeasurable.
- **Extra space:** two arrays of s longs = 16 bytes per slab (48 bytes for 3 slabs).
- **Easier to change:** the array design. Adding or removing a slab means editing one table row, not the program logic; the if version needs new code for each extra slab. The if version is slightly clearer to read for exactly three slabs.

Evidence: Working source code and a test table with expected and actual results for every required case, including roll-over, boundaries, invalid input and the hand-calculated total.

TASK 10 - Compare Expectations with Execution

A deterministic generator produces the units (0 to 400) for n households; each bill is computed with the tariff of Task 3. The same bills are added as float rupees, double rupees, 32-bit integer paise and 64-bit integer paise. The exact value is the 64-bit paise total.

Source code - t10.c

```
#include <stdio.h>
#include <stdint.h>
#include <limits.h>

static unsigned long long seed;
static int next_units(void)              /* deterministic pseudo-random 0..400 units */
{
    seed = seed * 6364136223846793005ULL + 1442695040888963407ULL;
    return (int)((seed >> 33) % 401);
}

static long compute_bill(int u)          /* paise */
{
    long p = 5000L;
    if (u > 100) p += (u > 200 ? 100 : u - 100) * 225L;
    if (u > 200) p += (long)(u - 200) * 450L;
    return p;
}

int main(void)
{
    int ns[] = {1000, 4000, 60000, 200000, 800000};

    printf("%8s %16s %16s %10s %16s %14s\n",
           "bills", "exact (paise)", "float (Rs)", "float err", "double err", "int32 total");

    for (int k = 0; k < 5; k++)
    {
        seed = 12345;
        float     tf = 0.0f;
        double    td = 0.0;
        long long tp = 0;
        uint32_t  t32 = 0;

        for (int i = 0; i < ns[k]; i++)
        {
            long p = compute_bill(next_units());
            tf  += p / 100.0f;
            td  += p / 100.0;
            tp  += p;
            t32 += (uint32_t)p;
        }
        double exact = tp / 100.0;
        printf("%8d %16lld %16.2f %10.2f %16.8f %14d\n",
               ns[k], tp, (double)tf, (double)tf - exact, td - exact, (int32_t)t32);
    }

    printf("\nINT_MAX paise  = %d  (Rs %d.%02d)\n", INT_MAX, INT_MAX / 100, INT_MAX % 100);
    printf("LONG_MAX paise = %ld\n", LONG_MAX);
    printf("LLONG_MAX      = %lld\n", LLONG_MAX);

    double d = 9007199254740992.0;       /* 2^53 */
    printf("\ndouble: 2^53 + 1 == 2^53 ? %s\n", (d + 1.0 == d) ? "yes (precision lost)" : "no");
    return 0;
}
```

Output (terminal run, gcc 13.3, Linux x86-64) - measured totals

```
$ gcc -Wall -Wextra t10.c -o t10
$   bills    exact (paise)       float (Rs)  float err       double err    int32 total
     1000         40621775        406217.75       0.00       0.00000000       40621775
     4000        165312875       1653128.75       0.00       0.00000000      165312875
    60000       2507600325      25075954.00     -49.25       0.00000000    -1787366971
   200000        832294697      83197480.00  -31989.75       0.00000000     -266987617
   800000      33253957750     332822496.00  282918.50       0.00000000    -1105780618
INT_MAX paise  = 2147483647  (Rs 21474836.47)
LONG_MAX paise = 9223372036854775807
LLONG_MAX      = 9223372036854775807
double: 2^53 + 1 == 2^53 ? yes (precision lost)
```

Bills (n)	Exact total	float error	int (32-bit) total	long / long long
4,000	Rs 1,653,128.75	Rs 0.00	correct	correct
60,000	Rs 25,076,003.25	Rs -49.25	overflowed (negative total)	correct

Bills (n)	Exact total	float error	int (32-bit) total	long / long long
800,000	Rs 332,539,577.50	Rs +282,918.50	overflowed	correct

Observations and reasons

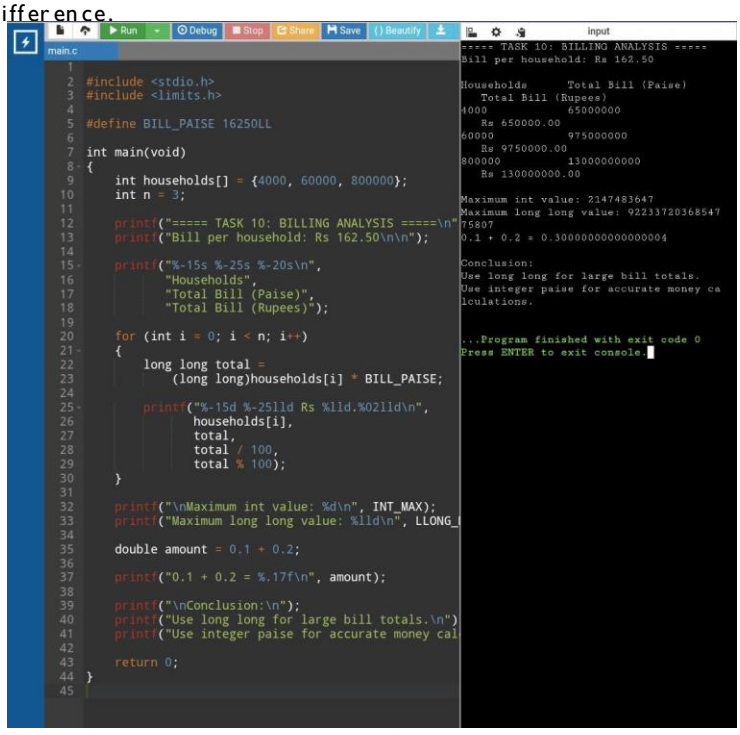
- **Float and paise totals:** agree to the paisa at 4,000 bills for this particular data, because most of these amounts are multiples of 0.25, which a binary float stores exactly (with bills such as Rs 162.35 the 4,000-bill total is already wrong by Rs 5.00 - Task 2). At 60,000, 200,000 and 800,000 bills the float totals differ by Rs 49.25, Rs 31,989.75 and Rs 282,918.50: float has only 24 bits of precision, so once the running total passes a few million rupees each added bill is rounded to a coarse grid.
- **int overflow:** a 32-bit int holds at most 2,147,483,647 paise (Rs 21,474,836.47). The district total is Rs 33.25 crore (33,253,957,750 paise), and even 60,000 bills (2,507,600,325 paise) overflow, giving a negative total. long (8 bytes here) and long long hold it with room to spare (limit 9.22×10^{18}).
- **sizeof compared with the C standard minimums:** the standard guarantees at least 8 bits for char, 16 for short and int, 32 for long, 64 for long long, FLT_DIG >= 6 and DBL_DIG >= 10. This system gives short = 2, int = 4, long = 8, long long = 8 bytes, FLT_DIG = 6, DBL_DIG = 15. long is 8 bytes here (Linux, LP64) but only 4 bytes on Windows (LLP64), so long alone would overflow on such systems; long long is guaranteed 64 bits.
- **Reason for any difference from expectation:** the only surprises were in the float column (growth of error with n) and the early int overflow at 60,000 bills rather than at the district size - both caused by the total being much larger than a single bill.

Alternative Concept and Complexity Analysis

If the district total were accumulated in a double (as paise or rupees): an integer total in a double is exact up to $2^{53} = 9.007 \times 10^{15}$ (measured: $2^{53} + 1$ equals 2^{53}). Our district total of 3.3×10^{10} paise is far below this limit, so a double would not lose accuracy at 800,000 bills if the total were kept in whole paise. If it is kept in fractional rupees the representation error appears again (the double error column in the table is below 10^{-8} rupees, but not guaranteed to be zero).

	long long	double
How it fails	Overflow: wraps around (signed overflow is undefined behaviour) at 9.22×10^{18} paise - a sudden wrong result	Precision loss: gradually drops the low digits above 2^{53}; error is small but silent
Time per addition	O(1), one integer add	O(1), one floating-point add
Space per variable	8 bytes	8 bytes

Evidence: Measured totals for 4,000, 60,000 and 800,000 bills for float, double, int and long long, with reasons for each difference.



TASK 11 - Recommend and Defend the Final Design

11.1 Recommended types, constants and preprocessor features

Item	Recommendation	Evidence from my experiments
Meter readings	unsigned short (typedef meter_t)	Matches the 16-bit register; (unsigned short)(current - previous) gives 256 for 65400 to 120 (Task 2)
Units	unsigned int	Never negative; at most 65,535
Bill and energy charge	integer paise in long (int64_t for portability)	Exact to the paisa; no float error (Rs 5.00 error with float in Task 2)
District total	long long (or int64_t)	Rs 33.25 crore needs 3.3 x 10^{10}; int overflowed at 60,000 bills (Task 10)
Tariff values	#define SLAB*_UNITS, RATE*_PAISE, FIXED_CHARGE_PAISE in billing.h	RATE2 changed to 230 in one line and every bill changed consistently (Task 3)
Macros	Fully parenthesised, or static inline functions when arguments may have side effects	SQUARE(a+1) = 36, MAX(i++, j) double evaluation shown (Task 4)
Structure	billing.h with include guards, billing.c, main; #ifdef DEBUG	DEBUG build prints the slab details; release build contains 234 fewer code bytes (Task 5)
Input	Check the return value of scanf (or use fgets + strtol)	Letters and negative values rejected (Tasks 6 and 9)

11.2 How a tariff revision is carried out

Edit the values in the one tariff block in billing.h (for example RATE2_PAISE 225 to 230), recompile with gcc -Wall -Wextra, rerun the test table of Task 9 (expected values change only for bills above 100 units), and distribute the new executable. Because every rate is written once, a revision cannot be applied "partly", which was the fault of the spreadsheet.

11.3 Limitations

A tariff change needs a recompile and redistribution of the program.

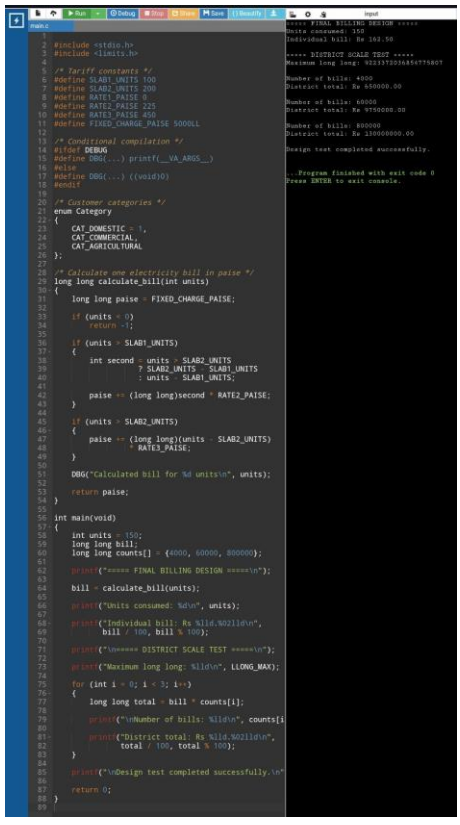
Exactly three slabs, one tariff for all categories, no taxes or arrears.

It assumes a 16-bit register, a single roll-over and monthly readings; a genuine meter fault looks like an invalid reading and needs manual checking.

long is only 32 bits on some platforms; int64_t / long long should be used for portability.

11.4 Is the design still correct at district scale?

Yes. A single bill is at most about 65,535 units, which is a few lakh rupees (under 3 x 10^7 paise) and fits easily in long. The district total measured 33,253,957,750 paise, about 277 million times smaller than the limit of long long. Only the district total needs the wide type; int would fail already at 60,000 houses.



Alternative Concept and Complexity Analysis

If the tariff were read from a configuration file at start-up (implemented in Task 12): the program opens the file and reads t values (9 values there), taking one file open plus t reads - microseconds, O(t) once per run - and the values occupy t x 8 = 72 bytes as variables. A revision is then a one-line text edit with no recompile or redistribution, whereas the compiled-in design needs a recompile and redeployment of the program on every revision (minutes, and the risk that some offices keep an old version). The price is that the file must be validated at start-up (missing or bad values), and the values can no longer be used as compile-time constants.

Evidence: Recommendations are tied to the measurements in Tasks 2, 3, 4, 5, 9 and 10 above.

TASK 12: Re-evaluate and Adapt the Design

A. Impact Analysis

What has changed from the original problem

	Original design	Changed requirement
Reading frequency	1 reading per month per house (12 a year)	1 reading every 15 minutes: 96 a day, 35,040 a year (2,920 times more data)
Unit of reading	whole units (kWh)	watt-hours: 1,000 times finer
Tariff	slab only	slab plus a time-of-day rule: 20% more between 6 PM and 10 PM, so each reading needs the hour
Tariff storage	compiled in with #define	changeable without recompiling, so read from a file
Money	whole paise always	Wh x paise per kWh gives fractions of a paisa (1 Wh at Rs 2.25/kWh = 0.225 paise)

Which data types are no longer sufficient

- **16-bit meter_t:** it holds only 65,535 Wh = 65.5 kWh, so a house would roll it over in a few days. The register must be 32-bit (uint32_t, 4,294,967,295 Wh); the roll-over correction (unsigned subtraction) is retained.
- **Yearly totals:** a house using 12,000 kWh a year is 1.2×10^7 Wh, which fits 32 bits, but 800,000 houses is 9.6×10^{12} Wh, which does not. Wh totals must be int64_t. The count of readings 35,040 x 800,000 = 2.8×10^{10} also exceeds 32 bits.
- **Money in whole paise:** no longer enough, because one reading can be worth a fraction of a paisa. Money is accumulated as an integer in finer units (paise x 10^5 = Wh x paise per kWh x percent) in int64_t, then rounded once per bill to whole paise. A yearly house value of about 6.5×10^{11} fits easily in int64_t, and rounding each bill before adding to the district total keeps that total well below the limit.

Which constants can no longer be fixed with #define

- **Now loaded at run time from the tariff file:** the two slab limits (in Wh), the three rates, the fixed charge, the peak start hour (18), the peak end hour (22) and the peak percentage (120). These are policy values the board wants to change without recompiling, so they live in a tariff_t structure filled at start-up and validated.
- **Remain #define:** READINGS_PER_DAY (96) and SCALE (100,000), because they are fixed physical or structural facts, not policy; the register size (32 bit) is a typedef.

Decision: MODIFY

The core of the design is retained: integer money, wrap-safe unsigned subtraction for the register, validation of the input and the header-organised structure. It is modified in two places: the types (32-bit register, 64-bit Wh and money in finer units) and the tariff, now a run-time structure instead of #define macros. The slab calculation is also changed from one formula per bill to a pass over the intervals in time order, because the 20% rule depends on the hour of each reading. A complete redesign is not needed.

Program: smart-meter billing with a tariff file

Source code - t12.c (compile: gcc -Wall -Wextra t12.c -o t12; run: ./t12 tariff.cfg)

```c
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>

typedef uint32_t wh_reg_t;          /* 32-bit cumulative watt-hour register */
typedef int64_t  wide_t;            /* wide accumulator: Wh and scaled money */

#define READINGS_PER_DAY 96         /* one reading every 15 minutes          */
#define SCALE            100000LL   /* Wh * paise/kWh * percent = paise*1e5  */

typedef struct {
    wide_t slab1_limit_wh, slab2_limit_wh;
    wide_t rate1, rate2, rate3;     /* paise per kWh */
    wide_t fixed_paise;
    wide_t peak_start, peak_end, peak_percent;
} tariff_t;

static int load_tariff(const char *file, tariff_t *t)
{
    struct { const char *key; wide_t *dst; } map[] = {
        {"slab1_limit_wh", &t->slab1_limit_wh}, {"slab2_limit_wh", &t->slab2_limit_wh},
        {"rate1_paise_per_kwh", &t->rate1}, {"rate2_paise_per_kwh", &t->rate2},
        {"rate3_paise_per_kwh", &t->rate3}, {"fixed_charge_paise", &t->fixed_paise},
        {"peak_start_hour", &t->peak_start}, {"peak_end_hour", &t->peak_end},
        {"peak_percent", &t->peak_percent}
    };
    int n = (int)(sizeof map / sizeof map[0]), found = 0;
    FILE *f = fopen(file, "r");
    if (!f) { printf("Cannot open tariff file %s\n", file); return 0; }

    char key[64]; long long v;
    while (fscanf(f, "%63s %lld", key, &v) == 2)
        for (int i = 0; i < n; i++)
            if (strcmp(key, map[i].key) == 0) { *map[i].dst = v; found++; }
    fclose(f);

    if (found != n || t->slab1_limit_wh >= t->slab2_limit_wh ||
        t->peak_start < 0 || t->peak_end > 24 || t->peak_percent < 100)
    { printf("Invalid tariff file\n"); return 0; }
    return 1;
}

/* scaled charge for one 15-minute interval; cum = Wh already used this month */
static wide_t charge_interval(const tariff_t *t, wide_t *cum, wh_reg_t e, int hour)
{
    wide_t mult = (hour >= t->peak_start && hour < t->peak_end) ? t->peak_percent : 100;
    wide_t remain = e, scaled = 0;

    while (remain > 0)
    {
        wide_t rate, room;
        if      (*cum < t->slab1_limit_wh) { rate = t->rate1; room = t->slab1_limit_wh - *cum; }
        else if (*cum < t->slab2_limit_wh) { rate = t->rate2; room = t->slab2_limit_wh - *cum; }
        else                               { rate = t->rate3; room = remain; }
```

```
        wide_t part = remain < room ? remain : room;
        scaled  += part * rate * mult;
        *cum    += part;
        remain  -= part;
    }
    return scaled;
}

int main(int argc, char *argv[])
{
    tariff_t t;
    if (argc < 2 || !load_tariff(argv[1], &t)) return 1;

    wh_reg_t reg = 4294960000u;                /* register close to its 32-bit limit */
    wide_t cum_wh = 0, peak_wh = 0, scaled = 0;
    int days = 30;

    for (int i = 0; i < days * READINGS_PER_DAY; i++)
    {
        int hour = (i % READINGS_PER_DAY) / 4;
        int peak = hour >= t.peak_start && hour < t.peak_end;
        wh_reg_t used = peak ? 300 : 100;      /* what the house really used   */
        wh_reg_t prev = reg;
        reg += used;                           /* register wraps past 4294967295 */
        wh_reg_t e = (wh_reg_t)(reg - prev);   /* wrap-safe interval energy      */

        if (peak) peak_wh += e;
        scaled += charge_interval(&t, &cum_wh, e, hour);
    }

    wide_t energy_paise = (scaled + SCALE / 2) / SCALE;   /* round ONCE, half-up */
    wide_t total = energy_paise + t.fixed_paise;

    printf("Tariff file       : %s\n", argv[1]);
    printf("Readings          : %d\n", days * READINGS_PER_DAY);
    printf("Total energy      : %lld Wh (%lld.%03lld kWh), peak %lld Wh\n",
           (long long)cum_wh, (long long)(cum_wh / 1000), (long long)(cum_wh % 1000),
           (long long)peak_wh);
    printf("Energy charge     : Rs %lld.%02lld\n", (long long)(energy_paise / 100), (long long)(energy_paise % 100));
    printf("Fixed charge      : Rs %lld.%02lld\n", (long long)(t.fixed_paise / 100), (long long)(t.fixed_paise % 100));
    printf("Total bill        : Rs %lld.%02lld\n", (long long)(total / 100), (long long)(total % 100));
    printf("Register wrapped  : %s (final register = %u)\n", reg < 4294960000u ? "yes" : "no", reg);
    return 0;
}
```

Source code - tariff.cfg (a text file; the program is not recompiled to change it)

```
slab1_limit_wh 100000
slab2_limit_wh 200000
rate1_paise_per_kwh 0
rate2_paise_per_kwh 225
rate3_paise_per_kwh 450
fixed_charge_paise 5000
peak_start_hour 18
peak_end_hour 22
peak_percent 120
```

Output (terminal run, gcc 13.3, Linux x86-64) - smart-meter run for 30 days (2,880 readings)

```
$ gcc -Wall -Wextra t12.c -o t12
$ ./t12 tariff.cfg
Tariff file       : tariff.cfg
Readings          : 2880
Total energy      : 384000 Wh (384.000 kWh), peak 144000 Wh
Energy charge     : Rs 1133.28
Fixed charge      : Rs 50.00
Total bill        : Rs 1183.28
Register wrapped  : yes (final register = 376704)
$ ./t12 tariff2.cfg      (rate2_paise_per_kwh changed from 225 to 230, no recompile)
Tariff file       : tariff2.cfg
Readings          : 2880
Total energy      : 384000 Wh (384.000 kWh), peak 144000 Wh
Energy charge     : Rs 1138.64
Fixed charge      : Rs 50.00
Total bill        : Rs 1188.64
Register wrapped  : yes (final register = 376704)
$ ./t12 tariff_bad.cfg    (incomplete file)
Invalid tariff file
```

Checks. The month has 30 x 96 = 2,880 readings; the house uses 100 Wh in a normal interval and 300 Wh in the 16 peak intervals of each day, so 30 x (80 x 100 + 16 x 300) = 384,000 Wh and the peak energy is 30 x 16 x 300 = 144,000 Wh, as printed. The register starts at 4,294,960,000 and wraps past 4,294,967,295 during the month ("Register wrapped: yes") and the bill is still correct. Without the peak rule the energy

charge would be 100 x 2.25 + 184 x 4.50 = Rs 1,053.00; with a 20% uplift on all of it the maximum would be Rs 1,263.60; the program gives Rs 1,133.28, inside this range, and the same figure was obtained with an independent calculation in Python. Changing the second rate to 230 in the file changes the bill to Rs 1,188.64 with no recompilation, and an incomplete file is rejected.

Evidence: Impact analysis (what changed, which types and constants no longer suffice), a justified MODIFY decision, and a working program with output.

